

Computational Intelligence

Laboratory Exercise 1: Fuzzy Systems

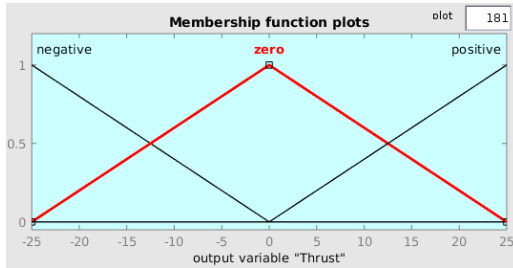
Josep de Cid, Gonzalo Recio

October 2019

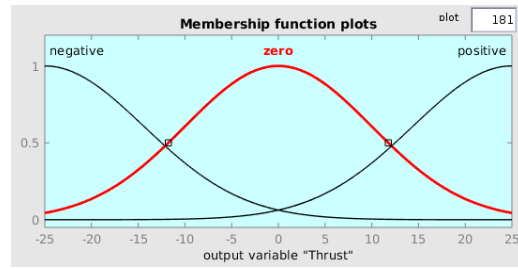
Membership functions

For the output variable Thrust we define 3 Membership Functions, labeled as *Negative*, *Zero* and *Positive*. For further experiments, we use 5 MFs, adding one extra membership function for *Negative* and *Positive* thrust, adding *Weak* and *Strong* options to both.

To choose the membership function in ANFIS, we need to look at the distribution of the data. Sadly, for **Mamdani**, trial and error is used because there is no exact method for choosing the MFs in literature. We are going to experiment with *Triangular* and *Gaussian* functions with the following distributions:



(a) Triangular MFs



(b) Gaussian MFs

The rule base defined is the following:

```
If (Error=Negative) and (ErrorDerivative=Decreasing) then (Thrust=Positive)
If (Error=Negative) and (ErrorDerivative=Stationary) then (Thrust=Positive)
If (Error=Negative) and (ErrorDerivative=Increasing) then (Thrust=Zero)
If (Error=Zero) and (ErrorDerivative=Decreasing) then (Thrust=Positive)
If (Error=Zero) and (ErrorDerivative=Stationary) then (Thrust=Zero)
If (Error=Zero) and (ErrorDerivative=Increasing) then (Thrust=Negative)
If (Error=Positive) and (ErrorDerivative=Decreasing) then (Thrust=Zero)
If (Error=Positive) and (ErrorDerivative=Stationary) then (Thrust=Negative)
If (Error=Positive) and (ErrorDerivative=Increasing) then (Thrust=Negative)
```

The logic behind these rules is the following: If the error is negative ($\theta_{desired} - \theta < 0$) and the derivative error is decreasing or stationary, it means that the pendulum will keep going to the negative direction, hence need to apply a positive thrust. If the error is negative but the derivative error is increasing, it will reach the equilibrium automatically, so we don't need to apply any thrust. The same logic is applied analogously to the 6 remaining rules.

Results

First executions with the proposed MFs distributions turned out to produce bad results. The Fuzzy Control System couldn't manage to stabilize the system to the expected $\theta_{desired}$. We thought that this could be caused because the membership function in the middle (which is in charge of firing **Thrust=Zero**) was too wide and the fuzzy control did not output a Thrust stabilization value different than zero when the error was less than 10. Therefore we tried another distribution for the MFs with a narrower MF for **Thrust=Zero** (middle one) and the results improved a lot.

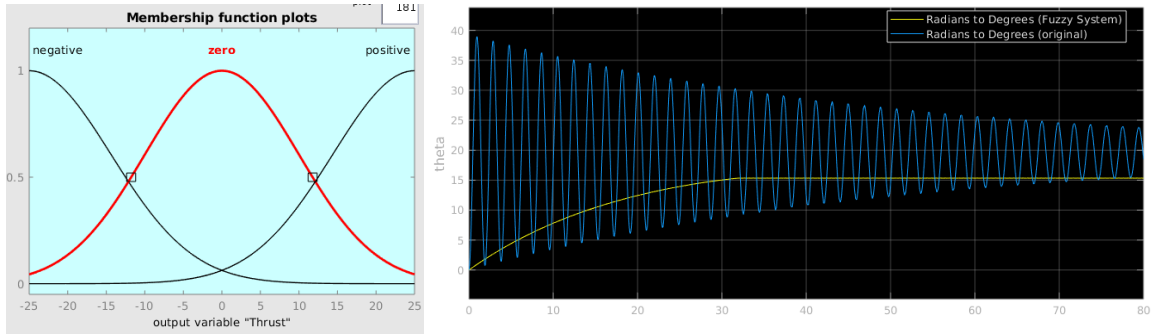


Figure 2: Undesired output with wide **Thrust=Zero** MF (not reaching $\theta_d = 20$)

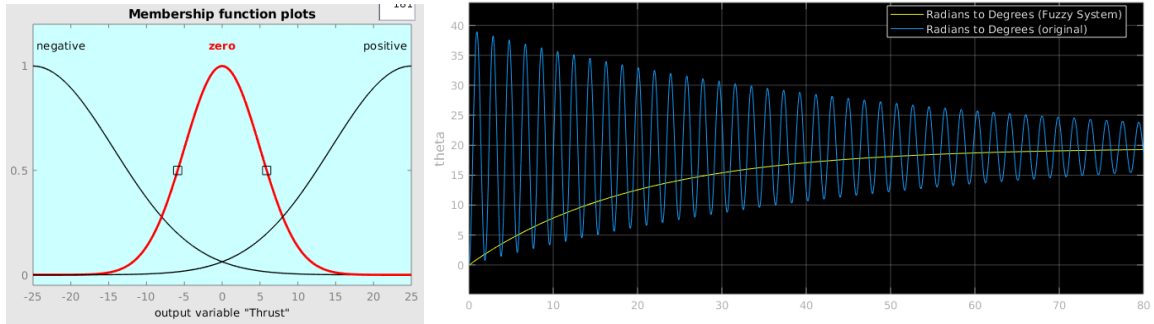
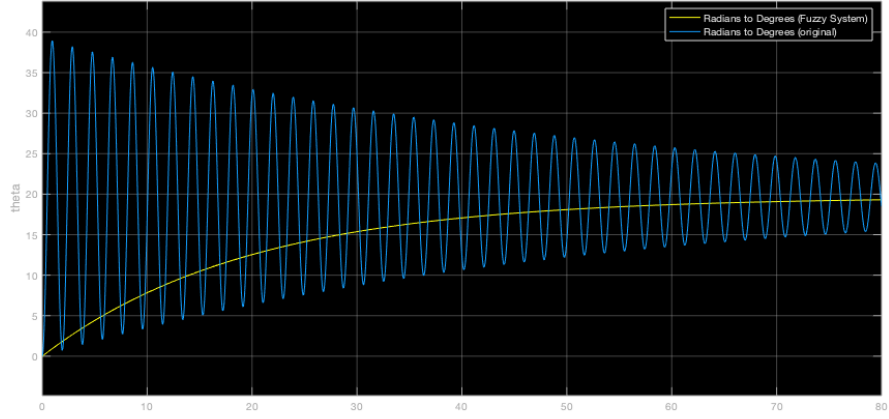


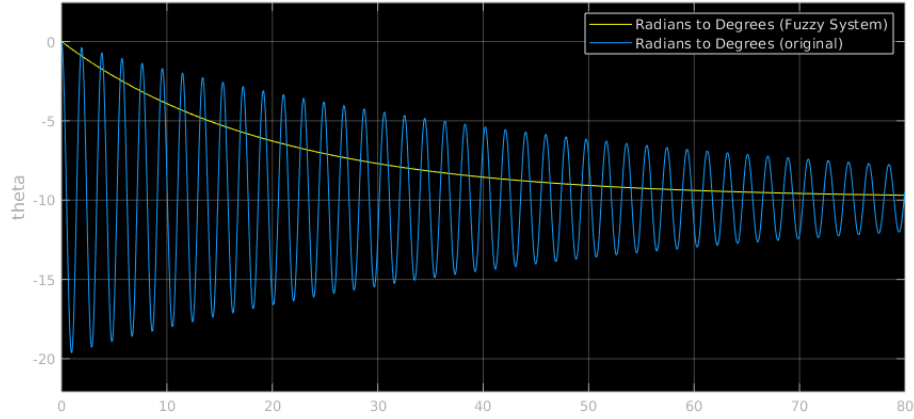
Figure 3: Improved output with narrow **Thrust=Zero** MF (stabilizing to almost $\theta_d = 20$)

The improvement using a narrower middle MF (representing **Zero** Thrust) is the same for both shape types of MF (*Triangular* and *Gaussian*) and the output obtained from the fuzzy control system also seems to be the same with both types. For this reason, we are going to select **Triangle** shaped MFs as they are simpler for doing the following experiments.

Execution results with *Triangular* shape MF



(a) $\theta = 20$ and $thrust = 0.123$ (3 triangle MFs)



(b) $\theta = -10$ and $thrust = -0.062$ (3 triangle MFs)

Figure 4: Open loop system (blue) vs. fuzzy control (yellow) for both scenarios.

More membership functions for the output

We wanted to see what would happen if we increase the number of membership functions of the output. For this problem, we decided to add to more functions to obtain more levels or options for the *thrust*, instead of all-or-nothing.

Now we are going to distinguish between **Strong_Positive** and **Weak_Positive** (and the same for **Negative** MF), the leading to a total of 5 membership functions. The following image show the distribution of the MFs and the shape we have chosen (as we justified before, we choose *Triangular* shaped MFs).

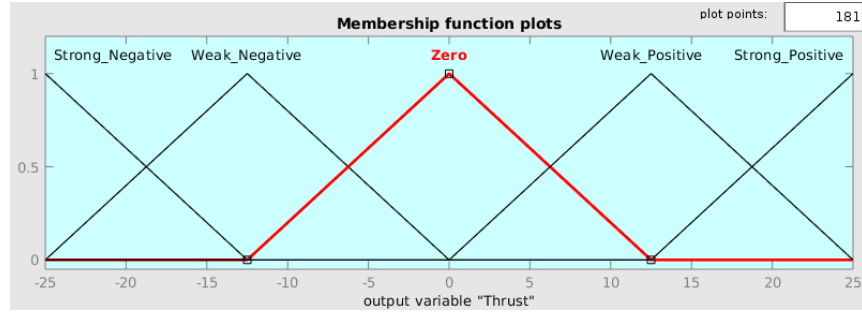


Figure 5: 5 *Triangular* MFs implemented for variable Thrust.

We define the new rules following the previous logic, but softening the thrust applied, being **Strong** if both errors are in the same direction, or **Weak** if one of them is 0.

```

If (Error=Negative) and (ErrorDerivative=Decreasing) then (Thrust=Strong_Positive)
If (Error=Negative) and (ErrorDerivative=Stationary) then (Thrust=Weak_Positive)
If (Error=Negative) and (ErrorDerivative=Increasing) then (Thrust=Zero)
If (Error=Zero) and (ErrorDerivative=Decreasing) then (Thrust=Weak_Positive)
If (Error=Zero) and (ErrorDerivative=Stationary) then (Thrust=Zero)
If (Error=Zero) and (ErrorDerivative=Increasing) then (Thrust=Weak_Negative)
If (Error=Positive) and (ErrorDerivative=Decreasing) then (Thrust=Zero)
If (Error=Positive) and (ErrorDerivative=Stationary) then (Thrust=Weak_Negative)
If (Error=Positive) and (ErrorDerivative=Increasing) then (Thrust=Strong_Negative)

```

Regarding the results, we did not see any significant improvement when using 5 MFs instead of 3 MFs well distributed (the results plots seem identical). Therefore, increasing the number of membership functions doesn't seem to affect the final result positively, as it looks like that using only 3 well distributed MFs is enough to cover all the situations for the output value.

However, we did not need to redistribute the 5 MFs to reach great results, so we can say that using some more MFs can help us to reach better results if we do not know how to properly distribute the few 3 initial MF to obtain better outputs.